

Programación II

Tema 4: Algoritmos y complejidad computacional

Departamento de Lenguajes y Sistemas Informáticos
Universitat Jaume I

Curso 2024/2025

Método recursivo

Método que se llama a sí mismo, directa o indirectamente.

Ejemplo de recursión directa

```
// Devuelve el sumatorio de 1 a n (n > 0)
public static long sumatorio(int n) {
    if (n == 1)
        return 1;
    else
        return sumatorio(n - 1) + n; // Llamada recursiva
}
```

Contenido

Recursión

Introducción a la complejidad algorítmica

Algoritmos de búsqueda y ordenación

Esquemas algorítmicos básicos

Ejemplo: factorial de un número

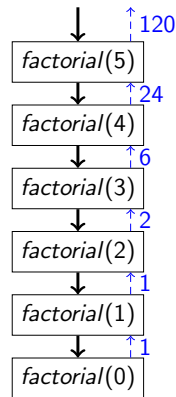
$$n! = \begin{cases} 1, & \text{si } n = 0 \\ (n-1)! \times n, & \text{si } n > 0 \end{cases}$$

Ejemplo: cálculo recursivo del factorial

```
// Devuelve el factorial de n (n >= 0)
public static long factorial(int n) {
    if (n == 0) // Caso base
        return 1;
    else
        return factorial(n - 1) * n;
}
```

Árbol de llamadas

Para la llamada `factorial(5)`:



Regresión infinita

Ejemplo: cálculo incorrecto del factorial

```
// Devuelve el factorial de n (n >= 0)
public static long factorial(int n) {
    if (n == 1)
        return 1;
    else
        return factorial(n - 1) * n;
}
```

¡Para `factorial(0)` no se alcanzaría el caso base!

⇒ `java.lang.StackOverflowError`

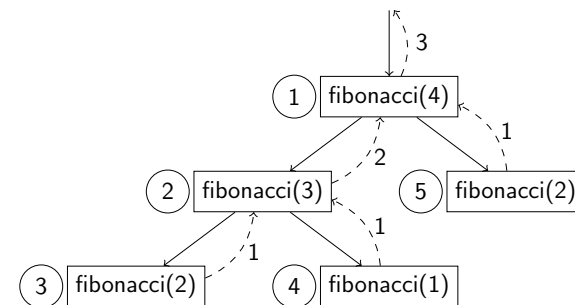
Ejemplo: n -ésimo número de Fibonacci

$$F_n = \begin{cases} 1, & \text{si } n = 1 \text{ o } n = 2 \\ F_{n-1} + F_{n-2}, & \text{si } n > 2 \end{cases}$$

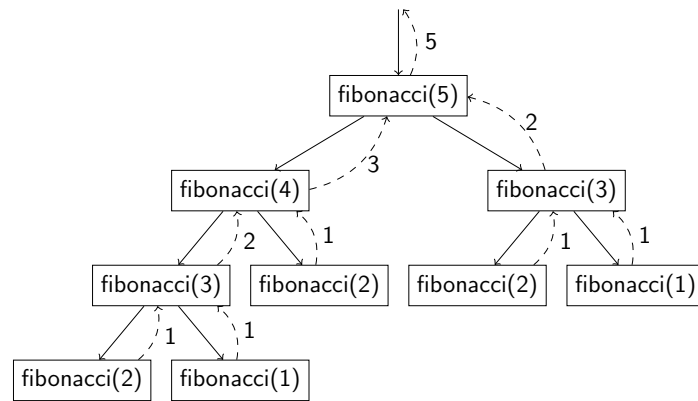
Ejemplo: cálculo recursivo del n -ésimo número de Fibonacci

```
// Devuelve el n-ésimo número de Fibonacci (n >= 1)
public static long fibonacci(int n) {
    if (n == 1 || n == 2) // Casos base
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Ejemplo: n -ésimo número de Fibonacci (cont.)

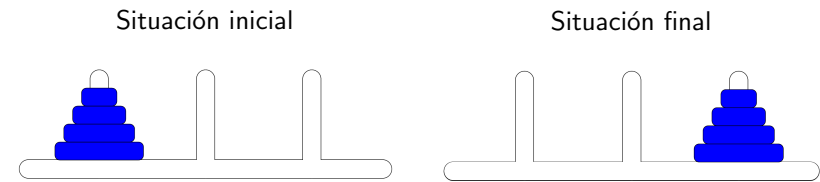


Ejemplo: n -ésimo número de Fibonacci (cont.)



¡Muy ineficiente!

Ejemplo: torres de Hanoi

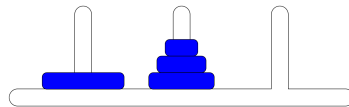


Restricciones:

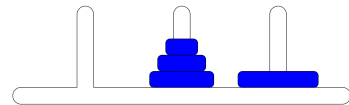
- ▶ En cada paso, mover el disco superior de una aguja a otra.
- ▶ No situar un disco sobre otro más pequeño.

Ejemplo: torres de Hanoi (cont.)

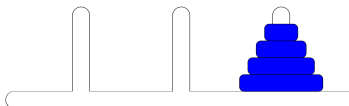
a) Mover $n-1$ discos de la aguja inicial a la libre:



b) Mover el último disco de la aguja inicial a la final:



c) Mover $n-1$ discos de la aguja libre a la final:



Ejemplo: torres de Hanoi (cont.)

```
public static void resuelveHanoi(int numDiscos,
    String inicio,
    String fin,
    String libre) {
    if (numDiscos == 1) {
        System.out.println("Mover disco de la aguja " +
            inicio + " a la aguja " + fin);
    } else {
        resuelveHanoi(numDiscos - 1, inicio, libre, fin);
        System.out.println("Mover disco de la aguja " +
            inicio + " a la aguja " + fin);
        resuelveHanoi(numDiscos - 1, libre, fin, inicio);
    }
}
```

Ejemplo: invertir una cola de cadenas

```
public static void invertirCola(ColaString cola) {
    if (cola.esVacía()) {
        return;
    } else {
        String primero = cola.extraerPrimero();
        invertirCola(cola);
        cola.insertar(primero);
    }
}
```

Conteo de pasos

- **Paso:** instrucción o conjunto de instrucciones cuyo tiempo de ejecución está acotado por un valor constante.

Ejemplo:

```
a = 10;
b = 2 * a + 1;
```

- **Coste computacional temporal:** número de pasos expresado en función de la talla del problema.

Introducción a la complejidad algorítmica

- Objetivo: comparar algoritmos en función de su **eficiencia**.
- Complejidad computacional (**coste**) de un algoritmo:
 - coste espacial
 - coste temporal
- Técnicas de estudio independientes de:
 - el ordenador
 - el lenguaje de programación
 - los detalles de implementación

Ejemplo: sumatorio de los n primeros naturales

Versión 1^a

```
def sumatorio(n):
    suma = 0
    for i in range(1, n + 1):
        suma = suma + i
    return suma
```

Versión 3^a

```
static int sumatorio(int n) {
    int suma = 0, i = 1;
    while (i <= n) {
        suma = suma + i;
        i = i + 1;
    }
    return suma;
}
```

Versión 2^a

```
static int sumatorio(int n) {
    int suma = 0;
    for (int i = 1; i <= n; i++)
        suma = suma + i;
    return suma;
}
```

El número de pasos crece en proporción directa a n

Ejemplo: sumatorio de los n primeros naturales (cont.)

Versión 4^a

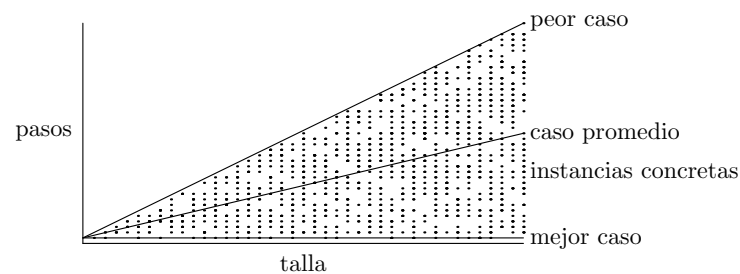
```
static int sumatorio(int n) {  
    return n * (n + 1) / 2;  
}
```

Versión 5^a

```
def sumatorio(n):  
    return n * (n + 1) // 2
```

El número de pasos es constante

Tiempo invertido en resolver cada instancia



- *acotado inferiormente* por una curva de la forma c_0
- *acotado superiormente* por una curva de la forma $c_1 \cdot n + c_2$

Coste dependiente de la instancia

Ejemplo: determinar si un vector contiene un elemento

```
public static boolean contiene(double[] v, double x) {  
    for (int i = 0; i < v.length; i++)  
        if (v[i] == x)  
            return true;  
    return false;  
}
```

- Mejor caso (x está en la primera posición de v): c_0
- Peor caso (x no está en v): $c_1 \cdot n + c_2$

Notación asintótica

Sea $f : \mathbb{N} \rightarrow \mathbb{R}^+$ una función de los naturales en los reales positivos.

Orden

$$O(f(n)) \doteq \{t : \mathbb{N} \rightarrow \mathbb{R}^+ \mid (\exists c \in \mathbb{R}^+, \exists n_0 \in \mathbb{N}), \forall n \geq n_0 \quad t(n) \leq cf(n)\}$$

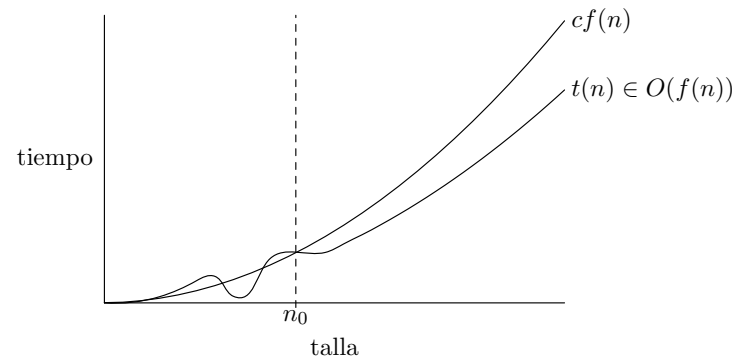
Omega

$$\Omega(f(n)) \doteq \{t : \mathbb{N} \rightarrow \mathbb{R}^+ \mid (\exists c \in \mathbb{R}^+, \exists n_0 \in \mathbb{N}), \forall n \geq n_0 \quad t(n) \geq cf(n)\}$$

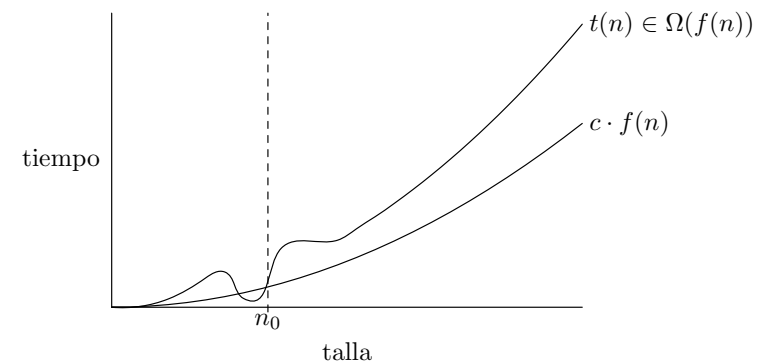
Zeta

$$\Theta(f(n)) \doteq O(f(n)) \cap \Omega(f(n))$$

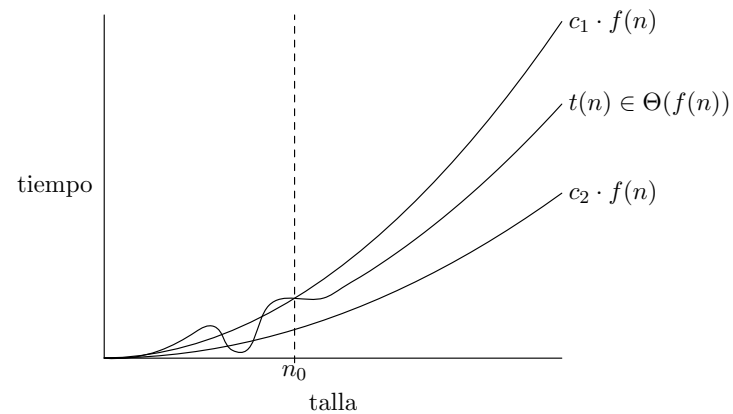
Cota asintótica superior (notación *orden de*)



Cota asintótica inferior (notación *omega*)



Cota asintótica superior e inferior (notación *zeta*)



Algunas relaciones entre órdenes usuales

$$O(1) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n^n)$$

Debemos indicar el orden de una función con su cota más ajustada

Propiedades de la notación O

$$O(c \cdot f(n)) = O(f(n)) \text{ para } c > 0$$

No importan las constantes

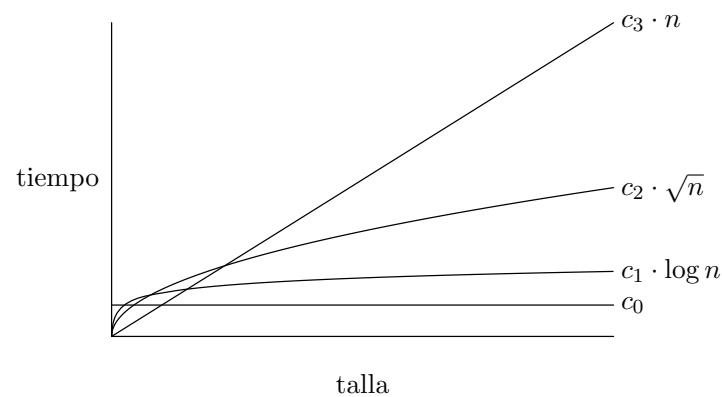
$$O(f(n) + g(n)) = O(\max\{f(n), g(n)\})$$

En las sumas nos quedamos con el mayor

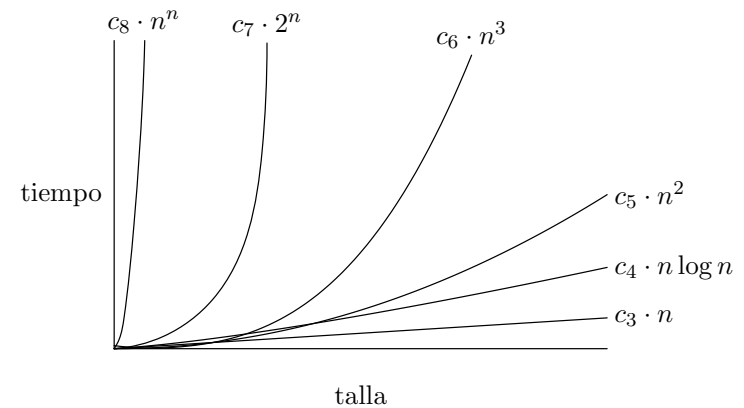
Ejemplos de uso de la notación O

Función	Notación O
$\log_3(n)$	$O(\log n)$
$5n^3$	$O(n^3)$
$n + \log(n)$	$O(n)$
$2n^2 + 3n + 1$	$O(n^2)$
$n \log(n) + n$	$O(n \log(n))$
$n^{100} + 2^n$	$O(2^n)$

Gráficas de crecimiento de funciones sublineales y lineal



Gráficas de crecimiento de funciones lineal y superlineales



Problema de la búsqueda estática

- ▶ Dado un valor x y un vector v , devolver la posición de x en v o una indicación de que no está.

v	4	3	7	7	13	17	16	15	9	12
	0	1	2	3	4	5	6	7	8	9

- ▶ Si x aparece más de una vez, devolver cualquier aparición.
- ▶ El vector no debe ser modificado.

Algoritmos de búsqueda

- ▶ Búsqueda secuencial (cuando el vector no esté ordenado)
- ▶ Búsqueda binaria (cuando el vector esté ordenado)

Búsqueda secuencial en un vector no ordenado

```
// Devuelve la posición de x en el vector
// o un valor negativo si no está
public static int buscar(int[] vector, int x) {
    for (int i = 0; i < vector.length; i++)
        if (vector[i] == x) // Encontrado
            return i;
    // No está
    return -1;
}
```

$O(n)$, donde n es la longitud del vector

Búsqueda secuencial en un vector ordenado

```
// Devuelve la posición de x en un vector ordenado
// de menor a mayor o un valor negativo si no está
public static int buscar(int[] vector, int x) {
    for (int i = 0; i < vector.length; i++)
        if (vector[i] == x)
            return i;
        else if (vector[i] > x) // Seguro que no está
            return -1;
    return -1; // No está
}
```

En promedio mejora, pero sigue siendo $O(n)$

Búsqueda binaria

```
public static int buscar(int[] vector, int x) {
    int inicio = 0;
    int fin = vector.length - 1;
    while (inicio <= fin) {
        int medio = (inicio + fin) / 2;
        if (vector[medio] < x)
            inicio = medio + 1; // Examinar mitad derecha
        else if (vector[medio] > x)
            fin = medio - 1; // Examinar mitad izquierda
        else // Encontrado
            return medio;
    }
    return -1; // No está
}
```

$O(\log n)$

Búsqueda binaria: número de iteraciones

En cada iteración, el espacio de búsqueda se reduce a $\approx$ la mitad:

Elementos iniciales:	n
Elementos tras la primera iteración:	menos de $\frac{n}{2}$
Elementos tras la segunda iteración:	menos de $\frac{n}{4}$
Elementos tras la tercera iteración:	menos de $\frac{n}{8}$
$\vdots$	$\vdots$
Elementos tras la k – ésima iteración:	menos de $\frac{n}{2^k}$

$\Rightarrow O(\log n)$ iteraciones

Búsqueda binaria: versión recursiva

```
public static int buscar(int[] vector, int x) {
    return buscar(vector, x, 0, vector.length - 1);
}

private static int buscar(int[] v, int x,
    int inicio, int fin) {
    if (inicio > fin) // No está
        return -1;
    int medio = (inicio + fin) / 2;
    if (v[medio] < x)
        return buscar(v, x, medio + 1, fin); // Mitad derecha
    else if (v[medio] > x)
        return buscar(v, x, inicio, medio - 1); // Mitad izquierda
    else // Encontrado
        return medio;
}
```

Algoritmos de ordenación

- Ordenación por selección
- Ordenación por inserción
- Ordenación por fusión o mezcla (*mergesort*)

Ordenación por selección

```
public static void ordenarSelección(int[] vector) {
    for (int i = 0; i < vector.length - 1; i++) {
        // Buscar posición del menor elemento entre i y el final
        int posMenor = i;
        for (int j = i + 1; j < vector.length; j++)
            if (vector[j] < vector[posMenor])
                posMenor = j;

        // Intercambiar elementos en posiciones i y posMenor
        int aux = vector[i];
        vector[i] = vector[posMenor];
        vector[posMenor] = aux;
    }
}
```

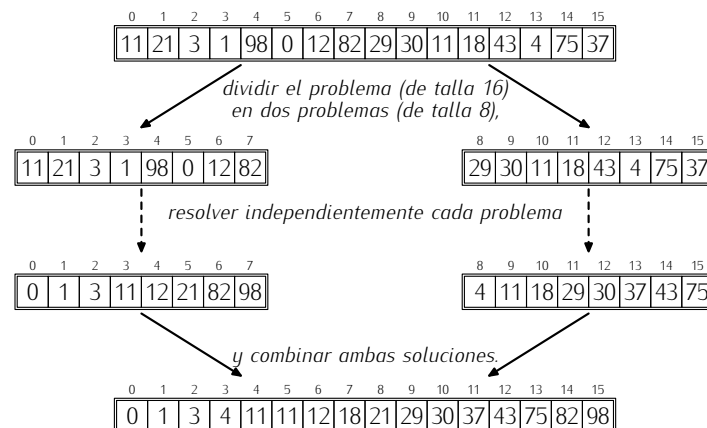
$O(n^2)$

Ordenación por inserción

```
public static void ordenarInserción(int[] vector) {
    for (int i = 1; i < vector.length; i++) {
        int temp = vector[i];
        int j = i - 1;
        while (j >= 0 && temp < vector[j]) {
            vector[j + 1] = vector[j];
            j--;
        }
        vector[j + 1] = temp;
    }
}
```

$O(n^2)$

Ordenación por fusión: la idea



Ordenación por fusión: una primera implementación

```
public static void ordenarFusión(int[] v) {
    if (v.length > 1) {
        // Dividir el vector en dos mitades
        int medio = v.length / 2;
        int[] mitad1 = Arrays.copyOfRange(v, 0, medio);
        int[] mitad2 = Arrays.copyOfRange(v, medio, v.length);

        // Ordenar cada mitad por separado
        ordenarFusión(mitad1);
        ordenarFusión(mitad2);

        // Mezclar las mitades ordenadas
        int[] mezcla = mezclarVectores(mitad1, mitad2);
        for (int i = 0; i < v.length; i++)
            v[i] = mezcla[i];
    }
}
```

$O(n \log n)$

Algoritmo de mezcla de vectores ordenados

```
public static int[] mezclarVectores(int[] v1, int[] v2) {
    int[] v3 = new int[v1.length + v2.length];
    int i1 = 0, i2 = 0, i3 = 0;
    // Repetir mientras queden datos en los dos vectores.
    while (i1 < v1.length && i2 < v2.length)
        if (v1[i1] < v2[i2])
            v3[i3++] = v1[i1++];
        else
            v3[i3++] = v2[i2++];
    // Copiar los elementos que queden en v1 o en v2.
    while (i1 < v1.length)
        v3[i3++] = v1[i1++];
    while (i2 < v2.length)
        v3[i3++] = v2[i2++];
    return v3;
}
```

$O(n)$

Esquemas algorítmicos

Estrategias de resolución de problemas:

- ▶ Divide y vencerás
- ▶ Estrategia voraz
- ▶ Búsqueda con retroceso
- ▶ Ramificación y acotación
- ▶ Programación dinámica

Divide y vencerás

Estrategia consistente en:

- ▶ Dividir un problema en una serie de subproblemas del mismo tipo, pero de menor tamaño.
- ▶ Solucionar independientemente todos los subproblemas.
- ▶ Combinar las soluciones obtenidas para formar la solución del problema original.